Gradient computation for the inverse problem: a comparison of differentiation methods

Sipan Bareyan

June 22, 2026

1 Introduction

We study the following nonlinear differential equation on the periodic interval $[0, 2\pi]$:

$$-u''(x) + \alpha u(x)^3 + V(x)u(x) = f(x), \qquad u(0) = u(2\pi),\ u'(0) = u'(2\pi), \tag{1}$$

where $V(x)$ is a potential and $f(x)$ is a forcing. We are interested in the inverse problem of recovering $V(x)$ of a given $u_{target}(x)$.

We compare methods for computing the gradient $g = \nabla_V M$ of the misfit $M(V) = \int_0^{2\pi} \big(u(V) - u_{target}\big)^2 dx$, where the state $u(V)$ solves the discretized nonlinear equation $F(u, V) = 0$ by Newton's method. The state solve and the adjoint construction are implemented in the library; we study only the *compute cost*, *memory*, and *accuracy* of the gradient, and how each scales with the number of parameters N. We contrast finite differences, automatic differentiation of the loss (forward and reverse), the hand-written adjoint, and AD equipped with a rule for the state solve, across the Julia AD libraries ForwardDiff, Zygote and Mooncake.

2 Methods

The loss maps the N entries of the discretized potential V to a scalar, so its gradient has one component per entry. The question that matters is how the cost of computing it grows with N.

- **Finite differences (FD).** Perturb each entry in turn: N+1 state solves, accuracy limited to $\sqrt{\varepsilon_{\text{mach}}}$ by the step-size trade-off. Baseline.
- **Forward-mode AD.** One tangent direction per pass, so N passes. It propagates a dual number through the Newton iteration rather than unrolling it to scalars, so a pass costs a constant times one state solve: machine-accurate for an exact solver, but same order as FD in this case of iterative solvers. $\mathcal{O}(N)$ like FD.
- **Reverse-mode AD.** A single backward pass returns the whole gradient, but it traces the Newton iteration and stores every intermediate state, so memory grows with the iteration count.
- **Adjoint (analytic).** One extra linear solve with the Newton-step operator at the converged solution (derived below): a fixed number of solves, independent of N, comparable to a single state solve. This is the reference.
- **Rule-based AD.** A custom rule for the *state solve* makes the AD tool insert the analytic adjoint **a pullback for** $u(V)$ instead of differentiating the Newton iteration. Because the rule sits on the solver rather than on our particular loss, it is reusable and composes inside any larger differentiable pipeline, while running on the production solver.

The adjoint gradient. At a converged state, $F(u(V), V) = 0$. Differentiating in V and writing $L = \partial F/\partial u$ for the (self-adjoint) Newton operator at u and $\partial F/\partial V = \mathrm{diag}(u)$ (the term Vu is linear in V),

$$L\,\delta u + \mathrm{diag}(u)\,\delta V = 0 \quad \Longrightarrow \quad \delta u = -L^{-1}\,\mathrm{diag}(u)\,\delta V.$$

The misfit varies as $\delta M = \frac{4\pi}{N}\langle u - u_0, \delta u\rangle$. Since L is HPD, L^{-1} is self-adjoint, so with the adjoint solve $L\lambda = u - u_0$,

$$\delta M = -\tfrac{4\pi}{N}\langle L^{-1}(u - u_0), \mathrm{diag}(u)\,\delta V\rangle = \big\langle -\tfrac{4\pi}{N}\,\mathrm{diag}(u)\,\lambda,\ \delta V\big\rangle.$$

Hence $\nabla_V M = -\frac{4\pi}{N}\,\mathrm{diag}(u)\,\lambda$: a single HPD (Krylov) solve at the converged u, independent of N.

Differentiating the linear solve. Every reverse-mode path carries a custom rule for the inner conjugate-gradient solve, so the linear system is never differentiated through; forward mode simply propagates duals through the Newton solver and the CG. The naive-versus-ruled comparison then measures the cost of unrolling Newton against the cost of replacing it by the adjoint rule.

3 Theoretical complexity

Let N be the number of parameters (grid points), n_N the number of Newton iterations, and n_c the number of CG iterations per linear solve. One spectral operator application costs $\mathcal{O}(N \log N)$, so one preconditioned CG solve costs $T_{\mathrm{lin}} = \mathcal{O}(n_c\, N \log N)$ and one state (Newton) solve costs $T_{\mathrm{sol}} = n_N\, T_{\mathrm{lin}}$. We express each gradient cost in these units; table 1 summarizes.

Table 1: Cost of one gradient evaluation. $T_{\mathrm{sol}} = n_N\, T_{\mathrm{lin}}$ is one state solve, $T_{\mathrm{lin}} = \mathcal{O}(n_c\, N \log N)$ one linear solve.

Method	State solves	Compute	Peak memory	Accuracy
Finite differences	N+1	$\mathcal{O}(N\, T_{\mathrm{sol}})$	$\mathcal{O}(N)$	$\mathcal{O}(\sqrt{\varepsilon_{\mathrm{mach}}})$
Forward AD (duals)	$\sim N$	$\mathcal{O}(N\, T_{\mathrm{sol}})$	$\mathcal{O}(N)$	machine, at iterate
Reverse AD naive (CG-ruled)	1+ bwd	$\mathcal{O}(T_{\mathrm{sol}})$	$\mathcal{O}(n_N N)$	machine, at iterate
Adjoint	1	$\mathcal{O}(T_{\mathrm{sol}})$	$\mathcal{O}(N)$	exact, at $u^\star$
Rule-based AD	1	$\mathcal{O}(T_{\mathrm{sol}})$	$\mathcal{O}(N)$	= adjoint

Within the cheap group. What separates the $\mathcal{O}(1)$-solve methods is the constant and the memory. The hand adjoint adds one linear solve and stores nothing extra. Naive reverse-mode redoes the iteration backward - $\approx n_N$ extra linear solves ($\approx 2\times$ the adjoint) and a tape of the n_N Newton iterates ($\mathcal{O}(n_N N)$ memory). Rule-based AD injects exactly that adjoint, recovering the single-solve cost and $\mathcal{O}(N)$ memory while staying composable.

4 Protocol

We fix a well-conditioned instance:

1. a forcing with nonzero mean, so the state stays bounded away from zero
2. a single evaluation point V_0.

We measure: [1]

1. *Time* is the minimum over warmed-up samples (BenchmarkTools, JIT excluded);
2. *allocations* is the total bytes allocated per call : cumulative heap churn over the call

[1] 13th Gen Intel(R) Core(TM) i7-1360P (16 threads); Julia 1.12.6, BLAS threads 8, ForwardDiff 1.3.3, Mooncake 0.5.29, Zygote 0.7.10; commit b3ff9fc, 2026-06-16.

3. *accuracy* is the relative ℓ_2 deviation from the analytic adjoint.

The study runs over $N = 5, \ldots, 500$. The AD tools are ForwardDiff (forward mode) and Mooncake and Zygote (reverse mode); the reverse-mode paths use the CG rule, and the rule-based paths add the state-solve rule.

The naive vs derived comparison. For the naive AD implementation, we avoid using fft and the Krylov library cg, which forces an unoptimized solve for a dense linear system differentiated through a dense $\mathcal{O}(N^2)$ reimplementation of the state solve. This keeps the Newton iteration AD-traceable. By contrast, FD, the adjoint, and the rule-based paths all use the production $\mathcal{O}(N \log N)$ spectral solver. The naive timings therefore carry a dense-algebra penalty on top of the differentiation cost and are not directly comparable to the production methods. The clean, like-for-like comparison is *adjoint versus rule-based*, both on the same production solver.

5 Results

Accuracy. Table 2 and fig. 1 report the relative ℓ_2 deviation of each gradient from the analytic adjoint reference at $N = 500$.

Table 2: Relative ℓ_2 deviation of each gradient from the analytic adjoint at $N = 500$.

category *String*	**tool** *String*	**mode** *String*	**rel_err** *Float64*
FD	-	-	4.42725e-6
adjoint	-	-	0.0
naive	ForwardDiff	forward	6.87036e-6
naive	Mooncake	reverse	1.97759e-6
naive	Zygote	reverse	1.97759e-6
ruled	Mooncake	reverse	5.57196e-12
ruled	Zygote	reverse	5.57196e-12

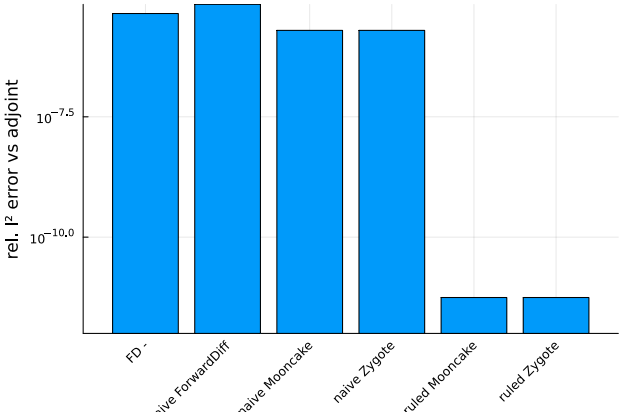

Figure 1: Relative ℓ_2 error of each method against the analytic adjoint (log scale); the adjoint itself ($= 0$, the reference) is omitted from the log axis.

The three inexact methods separate by mechanism. Finite differences sit near the step-size floor, $\sqrt{\varepsilon_{\text{mach}}} \approx 10^{-6}$. The extra error comes from the fact that we differentiate through a solver that is not perfect, so does not give exact analytical solution. Naive AD paths differ by how they treat the inner solve (§2): reverse mode carries the exact CG rule, so its only error is the *outer* Newton iterate gap - it differentiates the 100-step iterate, not the converged u — giving $\approx 2 \times 10^{-6}$; forward mode has no such rule and propagates duals through the truncated CG, adding a *CG-truncation* term to the tangent on top of that same iterate gap, hence the larger $\approx 7 \times 10^{-6}$. Rule-based AD bypasses both: it injects the analytic adjoint and lands at $\approx 5 \times 10^{-12}$ for both libraries — the linear-solve tolerance — reproducing the adjoint rather than the iterate.

Cost at fixed size. Table 3 gives wall time and allocations per gradient evaluation at $N = 500$.

Table 3: Time and allocations per gradient evaluation at fixed N.

category *String*	tool *String*	mode *String*	N *Int64*	time_s *Float64*	alloc_bytes *Int64*	rel_err *Float64*
FD	-	-	500	5.50531	13370617208	4.42725e-6
adjoint	-	-	500	0.00797893	27612248	0.0
naive	ForwardDiff	forward	500	86.9046	128850421472	6.87036e-6
naive	Mooncake	reverse	500	12.7309	20974671856	1.97759e-6
naive	Zygote	reverse	500	0.306921	1618399336	1.97759e-6
ruled	Mooncake	reverse	500	0.0164108	54230416	5.57196e-12
ruled	Zygote	reverse	500	0.0081782	27459376	5.57196e-12

At $N = 500$ the adjoint takes $\sim$8 ms and allocates 28 MB. Rule-based Zygote is indistinguishable from it ($\sim$8 ms, 27 MB), while rule-based Mooncake is about 2$\times$ in both time and allocations. The naive paths are far heavier: naive Zygote is $\sim$30$\times$ slower and allocates 1.6 GB, and naive Mooncake and forward-mode AD reach tens of seconds, allocating $\sim$210 GB and $\sim$130 GB. Finite differences, needing $N+1$ solves, is $\sim$670$\times$ the adjoint.

Scaling in N. Figures 2 and 3 show time and allocations against N on log–log axes; the backing data is tabulated per grid size in the appendix, each table also reporting the relative ℓ_2 error against the analytic adjoint.

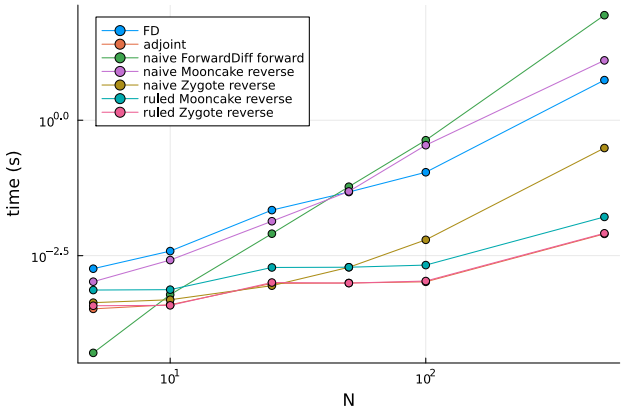

Figure 2: Time versus N.

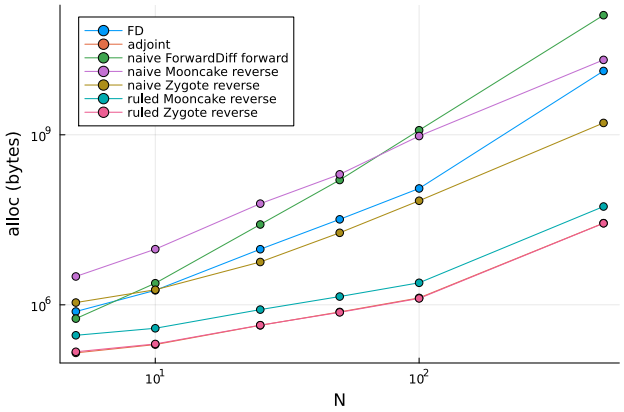

Figure 3: Allocations versus N.

The slopes confirm the two regimes: FD and forward-mode AD grow linearly in N (one solve per parameter), while the adjoint and both rule-based paths stay flat per gradient. In allocations, naive reverse-mode grows with its $\mathcal{O}(n_N N)$ tape, whereas the adjoint and rule-based paths stay flat. The adjoint is cheaper than FD across the whole range — already $\sim$6$\times$ at $N = 5$ — and the margin widens linearly with N.

6 Discussion

The measurements confirm table 1 and sharpen it into a clear recommendation. Finite differences, forward-mode AD and naive reverse-mode AD are all orders of magnitude slower and heavier than the adjoint — from $\sim$30$\times$ (naive Zygote), through $\sim$670$\times$ (FD) at $N = 500$ — so they serve only as baselines and correctness cross-checks. The adjoint and the rule-based paths compute the full gradient in $\mathcal{O}(1)$ solves. Among these, rule-based Zygote matches the hand adjoint in both time and allocations, while rule-based Mooncake trails at about 2$\times$.

Rule-based Zygote is therefore the method of choice for this problem: it recovers the analytic adjoint's speed, allocations and accuracy, yet stays a drop-in, composable rule on the production solver

rather than a bespoke hand derivation — which also makes Zygote the most effective Julia reverse-mode AD library here. Moreover, it directly inherits the ChainRules rules and is compatible with the rest of the Julia AD ecosystem. Enzyme is another option, but remains the most unreliable and needs a lot of configuration and debugging to work with naive configuration, or other complicated situations. Mooncake is a good option for reverse-mode AD, but it is not as fast as Zygote and has some limitations in terms of composability. The hand adjoint remains the conceptual reference, and naive differentiation is kept only to validate the rules.

References

[1] S. G. Johnson, *Notes on Adjoint Methods for 18.335*, MIT, 2006 (updated 2021).

[2] A. Levitt, *The Adjoint Trick*, 2020.

Appendix: statistics across N

The rule-based error is not constant across N: it tracks the adjoint solve's CG tolerance, growing from $\approx 3.5 \times 10^{-16}$ at $N = 5$ to $\approx 5.6 \times 10^{-12}$ at $N = 500$ once the solve becomes tolerance-limited.

Table 4: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 5$).

category *String*	**tool** *String*	**mode** *String*	**time_s** *Float64*	**alloc_bytes** *Int64*	**rel_err** *Float64*
FD	-	-	0.00181688	761280	1.15524e-7
adjoint	-	-	0.000329536	141968	0.0
naive	ForwardDiff	forward	5.0756e-5	570864	1.51471e-5
naive	Mooncake	reverse	0.00104616	3163848	5.26179e-8
naive	Zygote	reverse	0.000428973	1095872	5.26179e-8
ruled	Mooncake	reverse	0.000733333	289904	3.50732e-16
ruled	Zygote	reverse	0.000376294	148384	3.50732e-16

Table 5: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 10$).

category *String*	**tool** *String*	**mode** *String*	**time_s** *Float64*	**alloc_bytes** *Int64*	**rel_err** *Float64*
FD	-	-	0.0038393	1801072	1.8179e-7
adjoint	-	-	0.000392349	198912	0.0
naive	ForwardDiff	forward	0.000601701	2403760	1.64467e-5
naive	Mooncake	reverse	0.00262237	9582984	1.22867e-7
naive	Zygote	reverse	0.000485439	1842800	1.22867e-7
ruled	Mooncake	reverse	0.000745418	385504	3.53916e-16
ruled	Zygote	reverse	0.00038248	203792	3.53916e-16

Table 6: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 25$).

category *String*	**tool** *String*	**mode** *String*	**time_s** *Float64*	**alloc_bytes** *Int64*	**rel_err** *Float64*
FD	-	-	0.0219133	9567904	3.42596e-7
adjoint	-	-	0.000972388	434576	0.0
naive	ForwardDiff	forward	0.00804795	26142800	1.26784e-5
naive	Mooncake	reverse	0.0136601	60995584	1.33311e-7
naive	Zygote	reverse	0.000882954	5693792	1.33311e-7
ruled	Mooncake	reverse	0.00191381	823232	2.98367e-16
ruled	Zygote	reverse	0.00100916	437152	2.98367e-16

Table 7: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 50$).

category *String*	**tool** *String*	**mode** *String*	**time_s** *Float64*	**alloc_bytes** *Int64*	**rel_err** *Float64*
FD	-	-	0.0472682	32215600	5.58154e-7
adjoint	-	-	0.000989023	748624	0.0
naive	ForwardDiff	forward	0.0592777	159223304	1.26773e-5
naive	Mooncake	reverse	0.0484687	200426728	1.96469e-7
naive	Zygote	reverse	0.00192859	18648224	1.96469e-7
ruled	Mooncake	reverse	0.00193407	1398688	5.57772e-12
ruled	Zygote	reverse	0.000984608	736256	5.57772e-12

Table 8: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 100$).

category *String*	**tool** *String*	**mode** *String*	**time_s** *Float64*	**alloc_bytes** *Int64*	**rel_err** *Float64*
FD	-	-	0.109589	113004928	6.72626e-7
adjoint	-	-	0.00104042	1324416	0.0
naive	ForwardDiff	forward	0.427547	1202579944	1.2677e-5
naive	Mooncake	reverse	0.34605	951243992	1.9828e-6
naive	Zygote	reverse	0.00619767	68566784	1.9828e-6
ruled	Mooncake	reverse	0.00211795	2447400	5.57336e-12
ruled	Zygote	reverse	0.00107512	1295856	5.57336e-12

Table 9: Time, allocations and relative ℓ_2 error against the analytic adjoint ($N = 500$).

category *String*	**tool** *String*	**mode** *String*	**time_s** *Float64*	**alloc_bytes** *Int64*	**rel_err** *Float64*
FD	-	-	5.50531	13370617208	4.42725e-6
adjoint	-	-	0.00797893	27612248	0.0
naive	ForwardDiff	forward	86.9046	128850421472	6.87036e-6
naive	Mooncake	reverse	12.7309	20974671856	1.97759e-6
naive	Zygote	reverse	0.306921	1618399336	1.97759e-6
ruled	Mooncake	reverse	0.0164108	54230416	5.57196e-12
ruled	Zygote	reverse	0.0081782	27459376	5.57196e-12